

Web应用开发平时作业3

所有代码必须使用Python语言和Flask框架完成

在作业1的基础上修改代码，完成以下需求

1. 需要在浏览器中通过路由 `/order/<order_id>` 访问到订单小票页面。
2. 所有的数据需要从sqlite数据库中读取（不允许使用任何其他类型的数据库）。
3. 小票中的购买品项部分需要多于一种。如果作业1中只有一个品项，自行生成额外的数据，不用重新更换新的小票。
4. 在该小票下方，使用ECharts绘制一个简单的图形，显示该订单中各个品项的购买数量占比（饼图即可）。（或者使用其他图形，自己设计即可，数量不限）

注意

自行考虑数据库表结构，model等如何设计，完成功能即可。

可以不使用任何orm框架，需要提交建表的sql文件。

可以修改前端页面，跟实际越接近越好。

请上传作业1中相同的小票照片、所有相关代码文件（包括数据库文件，不要上传python虚拟环境文件）和运行成功的截图或视频。

GitHub代码仓库地址：<https://github.com/BillyNanLu/PythonWebFlask>

对应的作业代码地址：<https://github.com/BillyNanLu/PythonWebFlask/tree/main/week6/assignments>

运行截图

详情看附件运行视频！

代码

GitHub代码仓库地址：<https://github.com/BillyNanLu/PythonWebFlask/tree/main/week4/assignments>

数据库文件(schema.sql)

```
1  -- 创建用户表
2  CREATE TABLE users (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      name TEXT NOT NULL,
5      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
6  );
7
8  -- 创建商店表
9  CREATE TABLE stores (
10     id INTEGER PRIMARY KEY AUTOINCREMENT,
11     store_no TEXT NOT NULL,
```

```
12     store_name TEXT NOT NULL,
13     address TEXT
14 );
15
16 -- 创建产品表
17 CREATE TABLE products (
18     id INTEGER PRIMARY KEY AUTOINCREMENT,
19     name TEXT NOT NULL,
20     base_price REAL NOT NULL,
21     description TEXT
22 );
23
24 -- 创建订单表
25 CREATE TABLE orders (
26     id INTEGER PRIMARY KEY AUTOINCREMENT,
27     order_number TEXT NOT NULL UNIQUE,
28     user_id INTEGER NOT NULL,
29     store_id INTEGER NOT NULL,
30     order_time TIMESTAMP NOT NULL,
31     invoice_code TEXT,
32     FOREIGN KEY (user_id) REFERENCES users (id),
33     FOREIGN KEY (store_id) REFERENCES stores (id)
34 );
35
36 -- 创建订单项表
37 CREATE TABLE order_items (
38     id INTEGER PRIMARY KEY AUTOINCREMENT,
39     order_id INTEGER NOT NULL,
40     product_id INTEGER NOT NULL,
41     quantity INTEGER NOT NULL DEFAULT 1,
42     details TEXT,
43     FOREIGN KEY (order_id) REFERENCES orders (id),
44     FOREIGN KEY (product_id) REFERENCES products (id)
45 );
46
47 -- DP
48 CREATE TABLE dp (
49     id INTEGER PRIMARY KEY AUTOINCREMENT,
50     order_id INTEGER NOT NULL,
51     method TEXT NOT NULL,
52     code TEXT,
53     amount REAL,
54     details TEXT,
55     FOREIGN KEY (order_id) REFERENCES orders (id)
56 );
57
58 -- 创建支付表
59 CREATE TABLE payments (
60     id INTEGER PRIMARY KEY AUTOINCREMENT,
61     order_id INTEGER NOT NULL,
62     method TEXT NOT NULL,
63     code TEXT,
```

```

64     amount REAL DEFAULT 0.00,
65     discount_price REAL,
66     discount_details TEXT,
67     FOREIGN KEY (order_id) REFERENCES orders (id)
68 );
69
70
71
72 -- 插入示例数据
73 INSERT INTO users (name) VALUES ('BillyLu');
74 INSERT INTO users (name) VALUES ('GenieJT');
75
76 INSERT INTO stores (store_no, store_name, address)
77 VALUES ('55523', '临港港城新天地店', '上海市浦东新区茉莉路港城新天地225弄67号');
78 INSERT INTO stores (store_no, store_name, address)
79 VALUES ('54574', '星巴克臻选咖啡酒坊', '上海市黄浦区北京东路99号地上1楼L101A室');
80 INSERT INTO stores (store_no, store_name, address)
81 VALUES ('545XX', '上海烘焙工坊', '上海市静安区南京西路789号N110至N201单元');
82
83 INSERT INTO products (name, base_price, description) VALUES
84 ('星巴克经典咖啡混选中杯单杯MTDP', 24.90, ''),
85 ('星巴克星冰乐混选大杯单杯MTDP', 27.90, ''),
86 ('焦糖玛奇朵中杯', 34.00, '经典咖啡'),
87 ('焦糖玛奇朵大杯', 37.00, '经典咖啡'),
88 ('馥芮白中杯', 35.00, '经典咖啡'),
89 ('馥芮白大杯', 38.00, '经典咖啡'),
90 ('摩卡大杯', 36.00, '经典咖啡'),
91 ('摩卡中杯', 33.00, '经典咖啡'),
92 ('美式咖啡中杯', 27.00, '经典咖啡'),
93 ('美式咖啡大杯', 30.00, '经典咖啡'),
94 ('椰子丝绒燕麦拿铁中杯', 36.00, '经典咖啡'),
95 ('椰子丝绒燕麦拿铁大杯', 39.00, '经典咖啡'),
96 ('海盐焦糖风味冰镇浓缩中杯', 36.00, '经典咖啡'),
97 ('海盐焦糖风味冰镇浓缩大杯', 39.00, '经典咖啡'),
98 ('抹茶星冰乐中杯', 29.00, '星冰乐'),
99 ('抹茶星冰乐大杯', 32.00, '星冰乐'),
100 ('摩卡可可碎片星冰乐中杯', 33.00, '星冰乐'),
101 ('摩卡可可碎片星冰乐大杯', 36.00, '星冰乐'),
102 ('巧克力风味星冰乐中杯', 29.00, '星冰乐'),
103 ('巧克力风味星冰乐大杯', 32.00, '星冰乐'),
104 ('冰摇红莓黑加仑茶中杯', 23.00, '冰摇茶'),
105 ('冰摇红莓黑加仑茶大杯', 26.00, '冰摇茶'),
106 ('冰摇桃桃乌龙茶中杯', 29.00, '冰摇茶'),
107 ('冰摇桃桃乌龙茶大杯', 32.00, '冰摇茶'),
108 ('焙茶拿铁中杯', 26.00, '茶拿铁'),
109 ('焙茶拿铁大杯', 29.00, '茶拿铁'),
110 ('抹茶拿铁中杯', 26.00, '茶拿铁'),
111 ('抹茶拿铁大杯', 29.00, '茶拿铁'),
112 ('红茶拿铁中杯', 26.00, '茶拿铁'),
113 ('红茶拿铁大杯', 29.00, '茶拿铁'),
114 ('香烤白汁大虾芝士薄饼', 32.00, '烘培&三明治'),
115 ('滇香菌菇牛肉法棍三明治', 29.00, '烘培&三明治'),

```

```

116 ('烤法式火腿鸡蛋三明治', 23.00, '烘焙&三明治'),
117 ('培根芝士蛋堡', 25.00, '烘焙&三明治'),
118 ('火腿芝士可颂', 23.00, '烘焙&三明治'),
119 ('经典菠萝包', 16.00, '烘焙&三明治'),
120 ('千粹抹茶生巧蛋糕', 36.00, '蛋糕'),
121 ('经典提拉米苏蛋糕', 35.00, '蛋糕'),
122 ('经典瑞士卷', 29.00, '蛋糕');
123
124
125 -- billy 1
126 INSERT INTO orders (order_number, user_id, store_id, order_time, invoice_code)
127 VALUES ('7111', 1, 1, '2025-05-29 11:26:23', '212841aa2204eb39b6d20');
128 INSERT INTO order_items (order_id, product_id, quantity, details)
129 VALUES (1, 1, 1, '中/冰/焦糖玛奇朵'),
130         (1, 33, 1, '');
131 INSERT INTO dp (order_id, method, code, amount, details)
132 VALUES (1, 'Meituan', '770*****927', 1.00, 'DCT_lo1_MT activation');
133 INSERT INTO payments (order_id, method, code, amount, discount_price,
134                        discount_details)
135 VALUES (1, 'Alipay', '015*****035', 3.00, 0.03, '支付宝优惠:¥0.03');
136
137 -- billy 2
138 INSERT INTO orders (order_number, user_id, store_id, order_time, invoice_code)
139 VALUES ('7198', 1, 2, '2025-07-19 13:53:59', '215782aw2876cp43c6c99');
140 INSERT INTO order_items (order_id, product_id, quantity, details)
141 VALUES (2, 18, 1, ''),
142         (2, 6, 1, '');
143 INSERT INTO payments (order_id, method, code, amount, discount_price,
144                        discount_details)
145 VALUES (2, 'WeChat', '015*****955', 0.00, 0.00, '微信优惠:¥0.00');
146
147 -- GenieJT的订单1
148 INSERT INTO orders (order_number, user_id, store_id, order_time, invoice_code)
149 VALUES ('8234', 2, 3, '2025-06-15 09:45:12', '216395bc3789df56e7a12');
150 INSERT INTO order_items (order_id, product_id, quantity, details)
151 VALUES (3, 5, 1, '中/热/标准糖'),
152         (3, 31, 1, '加热');
153 INSERT INTO dp (order_id, method, code, amount, details)
154 VALUES (3, 'Eleme', '881*****345', 2.50, '新用户首单立减');
155 INSERT INTO payments (order_id, method, code, amount, discount_price,
156                        discount_details)
157 VALUES (3, 'WeChat', '026*****789', 57.50, 2.50, '微信支付优惠:¥2.50');
158
159 -- GenieJT的订单2
160 INSERT INTO orders (order_number, user_id, store_id, order_time, invoice_code)
161 VALUES ('8567', 2, 1, '2025-08-03 15:30:45', '217842de4567fg89h0i1');
162 INSERT INTO order_items (order_id, product_id, quantity, details)
163 VALUES (4, 16, 1, '大/少冰/少糖'),
164         (4, 38, 1, ''),

```

```

165         (4, 36, 1, '');
166     INSERT INTO payments (order_id, method, code, amount, discount_price,
167         discount_details)
168     VALUES (4, 'Alipay', '037*****123', 97.00, 5.00, '支付宝会员折扣:¥5.00');

```

数据模型(models.py)

```

1  from settings import db
2  from datetime import datetime
3
4
5  class User(db.Model):
6      __tablename__ = 'users'
7      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
8      name = db.Column(db.String(100), nullable=False)
9      created_at = db.Column(db.DateTime, default=datetime.utcnow)
10
11
12  class Store(db.Model):
13      __tablename__ = 'stores'
14      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
15      store_no = db.Column(db.String(20), nullable=False)
16      store_name = db.Column(db.String(100), nullable=False)
17      address = db.Column(db.String(200))
18
19
20  class Product(db.Model):
21      __tablename__ = 'products'
22      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
23      name = db.Column(db.String(100), nullable=False)
24      base_price = db.Column(db.Float, nullable=False)
25      description = db.Column(db.Text)
26
27
28  class Order(db.Model):
29      __tablename__ = 'orders'
30      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
31      order_number = db.Column(db.String(50), nullable=False, unique=True)
32      user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
33      store_id = db.Column(db.Integer, db.ForeignKey('stores.id'), nullable=False)
34      order_time = db.Column(db.DateTime, nullable=False)
35      invoice_code = db.Column(db.String(50))
36
37      # 关系映射
38      user = db.relationship('User', backref=db.backref('orders', lazy=True))
39      store = db.relationship('Store', backref=db.backref('orders', lazy=True))
40
41
42  class OrderItem(db.Model):
43      __tablename__ = 'order_items'
44      id = db.Column(db.Integer, primary_key=True, autoincrement=True)

```

```

45     order_id = db.Column(db.Integer, db.ForeignKey('orders.id'), nullable=False)
46     product_id = db.Column(db.Integer, db.ForeignKey('products.id'), nullable=False)
47     quantity = db.Column(db.Integer, default=1, nullable=False)
48     details = db.Column(db.Text)
49
50     # 关系映射
51     order = db.relationship('Order', backref=db.backref('items', lazy=True))
52     product = db.relationship('Product', backref=db.backref('order_items',
53 lazy=True))
54
55 class Dp(db.Model):
56     __tablename__ = 'dp'
57     id = db.Column(db.Integer, primary_key=True, autoincrement=True)
58     order_id = db.Column(db.Integer, db.ForeignKey('orders.id'), nullable=False)
59     method = db.Column(db.String(50), nullable=False)
60     code = db.Column(db.String(50))
61     amount = db.Column(db.Float)
62     details = db.Column(db.Text)
63
64     order = db.relationship('Order', backref=db.backref('dp_records', lazy=True))
65
66
67 class Payment(db.Model):
68     __tablename__ = 'payments'
69     id = db.Column(db.Integer, primary_key=True, autoincrement=True)
70     order_id = db.Column(db.Integer, db.ForeignKey('orders.id'), nullable=False)
71     method = db.Column(db.String(50), nullable=False)
72     code = db.Column(db.String(50))
73     amount = db.Column(db.Float, default=0.00)
74     discount_price = db.Column(db.Float)
75     discount_details = db.Column(db.Text)
76
77     order = db.relationship('Order', backref=db.backref('payments', lazy=True))

```

settings.py

```

1  from flask_sqlalchemy import SQLAlchemy
2  db = SQLAlchemy()
3
4  class Config:
5      DEBUG = True
6      SQLALCHEMY_DATABASE_URI = 'sqlite:///starbucks.db'
7      SQLALCHEMY_TRACK_MODIFICATIONS = False

```

App.py

```

1  from flask import Flask, render_template, request
2  from settings import Config, db
3  # 导入模型
4  from models import User, Store, Product, Order, OrderItem, Dp, Payment

```

```

5  from datetime import datetime
6
7  app = Flask(__name__)
8  config = Config()
9  app.config.from_object(config)
10 db.init_app(app)
11
12
13 # 路由: /order/<order_id> 显示单个订单详情
14 @app.route('/order/<int:order_id>')
15 def show_order(order_id):
16     # 查询订单信息
17     order = Order.query.get_or_404(order_id)
18     # 查询关联的商店信息
19     store = Store.query.get(order.store_id)
20     # 查询订单项
21     order_items = OrderItem.query.filter_by(order_id=order_id).all()
22     # 查询DP信息
23     dp = Dp.query.filter_by(order_id=order_id).first()
24     # 查询支付信息
25     payment = Payment.query.filter_by(order_id=order_id).first()
26
27     # 计算总金额
28     total_amount = sum(item.product.base_price * item.quantity for item in
order_items)
29     if dp:
30         total_amount -= dp.amount
31     if payment and payment.discount_price:
32         total_amount -= payment.discount_price
33
34     return render_template(
35         "Starbucks.html",
36         store_no=store.store_no,
37         store_name=store.store_name,
38         order=order,
39         order_items=order_items,
40         dp=dp,
41         payment=payment,
42         total_amount=total_amount
43     )
44
45
46 # 路由: /user/<user_id> 显示用户所有订单
47 @app.route('/user/<int:user_id>')
48 def user_orders(user_id):
49     user = User.query.get_or_404(user_id)
50     # 获取用户所有订单
51     orders = Order.query.filter_by(user_id=user_id).all()
52
53     # 为每个订单计算总金额并关联相关数据
54     order_list = []
55     for order in orders:

```

```

56     store = Store.query.get(order.store_id)
57     items = OrderItem.query.filter_by(order_id=order.id).all()
58     total = sum(item.product.base_price * item.quantity for item in items)
59     dp = Dp.query.filter_by(order_id=order.id).first()
60     if dp:
61         total -= dp.amount
62     payment = Payment.query.filter_by(order_id=order.id).first()
63     if payment and payment.discount_price:
64         total -= payment.discount_price
65
66     order_list.append({
67         'order': order,
68         'store': store,
69         'total': total,
70         'item_count': len(items)
71     })
72
73     return render_template(
74         "user_orders.html",
75         user=user,
76         order_list=order_list
77     )
78
79
80 if __name__ == "__main__":
81     app.run()

```

Starbucks.html

```

1  <!DOCTYPE html>
2  <html lang="zh-CN">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>星巴克购物小票</title>
7      <link rel="stylesheet" href="{{url_for('static',
8  filename='css/Starbucks.css')}}">
9  </head>
10 <body>
11     <div class="receipt">
12         <div class="logo">
13             
15             <span>STARBUCKS</span>
16         </div>
17         <div class="store-info">
18             <p>Starbucks Coffee China</p>
19             <p>{{store_no}}-{{store_name}}</p>
20             <p>外带</p>

```

```
21
22     <div class="order-number">{{order.order_number}}</div>
23
24     <div class="header-row">
25         <div class="header-item">商品</div>
26         <div class="header-item">数量</div>
27         <div class="header-item">金额</div>
28     </div>
29
30     <hr class="separator">
31
32     <!-- 循环展示所有商品 -->
33     {% for item in order_items %}
34     <div class="product">
35         <div class="product-name">
36             <div>{{item.product.name}}</div>
37             <div>¥{{ "%.2f" | format(item.product.base_price) }}</div>
38         </div>
39         <div class="product-details">
40             <div>{{item.details or '无备注'}}</div>
41             <div>×{{item.quantity}}</div>
42             <div>¥{{ "%.2f" | format(item.product.base_price * item.quantity)
}}</div>
43         </div>
44     </div>
45     {% endfor %}
46
47     <!-- 折扣信息 -->
48     {% if dp %}
49     <div class="discount">
50         <div>{{dp.details}}</div>
51         <div>-¥{{ "%.2f" | format(dp.amount) }}</div>
52     </div>
53     {% endif %}
54
55     <div class="total">总价 ¥{{ "%.2f" | format(total_amount) }}</div>
56
57     <hr class="separator">
58
59     <div class="info-row">
60         <div>下单时间</div>
61         <div>{{order.order_time.strftime('%Y-%m-%d %H:%M:%S')}}</div>
62     </div>
63
64     <div class="info-row">
65         <div>付款</div>
66         <div>¥{{ "%.2f" | format(total_amount) }}</div>
67     </div>
68
69     <div class="info-row">
70         <div>应付找零</div>
71         <div>¥0.00</div>
```

```
72     </div>
73
74     <!-- 支付方式信息 -->
75     <div class="payment-method">
76         <div class="info-row">
77             <div class="payment-method-name">{{payment.method}}</div>
78             <div class="payment-amount">¥{{ "%.2f" | format(payment.amount) }}
    </div>
79
80             <div class="payment-method-number">{{payment.code}}</div>
81         </div>
82
83         {% if dp %}
84         <div class="payment-method">
85             <div class="info-row">
86                 <div class="payment-method-name">{{dp.method}}</div>
87                 <div class="payment-amount">¥{{ "%.2f" | format(dp.amount or 0) }}
    </div>
88
89                 <div class="payment-method-number">{{dp.code}}</div>
90             </div>
91             {% endif %}
92
93         <hr class="separator">
94
95         <div class="payment-details">
96             <h3>{{payment.method}}支付详情</h3>
97
98             <div class="info-row">
99                 <div>用户支付User Paid</div>
100                 <div>¥{{ "%.2f" | format(payment.amount) }}</div>
101             </div>
102
103             {% if payment.discount_details %}
104             <div class="info-row">
105                 <div>{{payment.discount_details}}</div>
106             </div>
107             {% endif %}
108
109             <div class="info-row">
110                 <div>支付订单号</div>
111                 <div>{{payment.code}}</div>
112             </div>
113         </div>
114
115         <hr class="final-separator">
116
117         <div class="info-row">
118             <div>发票提取码</div>
119             <div>{{order.invoice_code}}</div>
120         </div>
121
```

```
122     <hr class="final-separator">
123
124     <!-- 饼图容器 -->
125     <div class="chart-container">
126         <h3>商品数量占比</h3>
127         <div id="pieChart" style="width: 100%; height: 300px;"></div>
128     </div>
129
130 </div>
131
132     <script type="text/javascript" src="/static/vendor/echarts-stat/ecStat.min.js">
</script>
133     <script src="/static/js/echarts.min.js"></script>
134     <script src="/static/js/infographic.js"></script>
135     <script src="/static/js/show_data_pie.js"></script>
136     <script src="/static/js/f_data.js"></script>
137     <script src="/static/js/show_column_data.js"></script>
138     <script src="/static/js/show_broken_line_data.js"></script>
139
140     <script>
141         // 准备饼图数据（从模板变量获取）
142         const pieData = [
143             {% for item in order_items %}
144             {
145                 name: "{{ item.product.name }}",
146                 value: {{ item.quantity }}
147             }{% if not loop.last %},{% endif %}
148             {% endfor %}
149         ];
150
151         // 初始化饼图
152         const chartDom = document.getElementById('pieChart');
153         const myChart = echarts.init(chartDom);
154         let option;
155
156         option = {
157             title: {
158                 // text: '订单商品数量占比',
159                 left: 'center'
160             },
161             tooltip: {
162                 trigger: 'item',
163                 formatter: "{a} <br/>{b}: {c} ({d}%)"
164             },
165             legend: {
166                 orient: 'vertical',
167                 left: 'left'
168             },
169             series: [
170                 {
171                     name: '购买数量',
172                     type: 'pie',
```

```
173         radius: '70%',
174         data: pieData,
175         emphasis: {
176             itemStyle: {
177                 shadowBlur: 10,
178                 shadowOffsetX: 0,
179                 shadowColor: 'rgba(0, 0, 0, 0.5)'
180             }
181         }
182     }
183 ]
184 };
185
186 option && myChart.setOption(option);
187
188 // 响应窗口大小变化
189 window.addEventListener('resize', function() {
190     myChart.resize();
191 });
192 </script>
193 </body>
194 </html>
```